

Java programming

[2021,2019,2018,2017,2016,2015,2014,2011]

8 year question paper solved

1] Explain the oops concepts in brief. [8marks] [2021,2019,2018,2017,2016,2015,2014,2011]

Java OOPs Concepts

- Object-Oriented Programming is a paradigm that provides many concepts, such as **inheritance**, **data binding**, **polymorphism**, etc.
- The programming paradigm where everything is represented as an object is known as a truly object-oriented programming language.
- Java strongly supports the concepts of oop.
- There are 4 types of access specifiers :
1) public 2) private 3) protected 4) default
- Message passing object communication with one another by sending and receiving information to each other.
- **Object** means a real-world entity such as a pen, chair, table, computer, watch, etc.
- It simplifies software development and maintenance by providing some concepts:
 - Object
 - Class
 - Inheritance
 - Polymorphism
 - Abstraction
 - Encapsulation

- Class : class is a user defined blueprint and collection of object.
- Object : any real world entity which has its own existence is known as object.
- Encapsulation : the capability to hide the data.
- Inheritance: inheritance is one of the most important and useful characteristics of OOPS language.
inheritance is the derivation of new class from old class.
- Polymorphism : poly means many, morphism means form. multiple forms are formed.
- Abstraction : data abstraction is the process of representing the essential features without including background details.

2] write short note on java history [4m] [2021,2019,2018,2017,2016,2015,2014]

- The history of Java is very interesting.
- Java was originally designed for interactive television, but it was too advanced technology for the digital cable television industry at the time. The history of Java starts with the Green Team.
- Java team members (also known as Green Team), initiated this project to develop a language for digital devices such as set-top boxes, televisions, etc. However, it was best suited for internet programming. Later, Java technology was incorporated by Netscape.
- The principles for creating Java programming were "Simple, Robust, Portable, Platform-independent, Secured, High Performance, Multithreaded, Architecture Neutral, Object-Oriented, Interpreted, and Dynamic".
- Java was developed by James Gosling, who is known as the father of Java, in 1995.
- James Gosling and his team members started the project in the early '90s.

Bca expert

- Currently, Java is used in internet programming, mobile devices, games, e-business solutions, etc.

3] Define web page . Explain the characteristics of java [8m] [2019, 2018 ,2017,2016,2015,2011]

A web page is a single hypertext document available on World Wide Web (WWW). It is composed of HTML elements and displayed on the user's browser.

A webpage is a document written in HTML and can be viewed on any web browser. It is contained within the web server, which can be accessed by entering the URL for that web page, and once it is loaded, it appears on the user's web browser.

the most important features of the Java language is given below.

1. Simple
2. Object – oriented
3. Portable
4. Platform independent
5. Secured
6. Robust
7. Architecture neutral
8. Interpreted
9. High performance
10. Multithreaded
11. distributed

Simple

- Java is very easy to learn, and its syntax is simple, clean and easy to understand.
- According to Sun Microsystem, Java language is a simple programming language because:

Object-oriented

- java is an object –oriented programming language.
- Everything in Java is an object. Object-oriented means we organize our software as a combination of different types of objects that incorporate both data and behavior.

Platform Independent

- Java is platform independent because it is different from other languages like C.
- A platform is the hardware or software environment in which a program runs.
- Java provides a software-based platform.

Secured

- Java is best known for its security. With Java, we can develop virus-free systems. Java is secured because
- Java language provides these securities by default. Some security can also be provided by an application developer explicitly through SSL, JAAS, Cryptography, etc.

Robust

- java is robust because:
- It uses strong memory management.
- There is a lack of pointers that avoids security problems.
- Java provides automatic garbage collection which runs on the Java Virtual Machine to get rid of objects which are not being used by a Java application anymore.
- There are exception handling and the type checking mechanism in Java. All these points make Java robust.

Portable

- Java is portable because it facilitates you to carry the Java bytecode to any platform. It doesn't require any implementation.

High-performance

- Java is faster than other traditional interpreted programming languages because Java bytecode is "close" to native code.
- It is still a little bit slower than a compiled language (e.g., C++).
- Java is an interpreted language that is why it is slower than compiled languages, e.g., C, C++, etc.

Distributed

- Java is distributed because it facilitates users to create distributed applications in Java.
- RMI and EJB are used for creating distributed applications.
- This feature of Java makes us able to access files by calling the methods from any machine on the internet.

Multi-threaded

- A thread is like a separate program, executing concurrently.
- We can write Java programs that deal with many tasks at once by defining multiple threads.
- The main advantage of multi-threading is that it doesn't occupy memory for each thread.
- It shares a common memory area.
- Threads are important for multi-media, Web applications, etc.

Dynamic

- Java is a dynamic language. It supports the dynamic loading of classes. It means classes are loaded on demand.
- It also supports functions from its native languages, i.e., C and C++.

4] How java program is compiled and interpreted ? Explain with example [8m] [2019,2018]

- Java is completely portable; the same Java code will run identically on different platforms, regardless of hardware compatibility or operating systems.
- The Java source code first compiled into a binary byte code using Java compiler, then this byte code runs on the JVM (Java Virtual Machine), which is a software based interpreter.
- So Java is considered as both interpreted and compiled.
- The compiled byte code allows JVM.

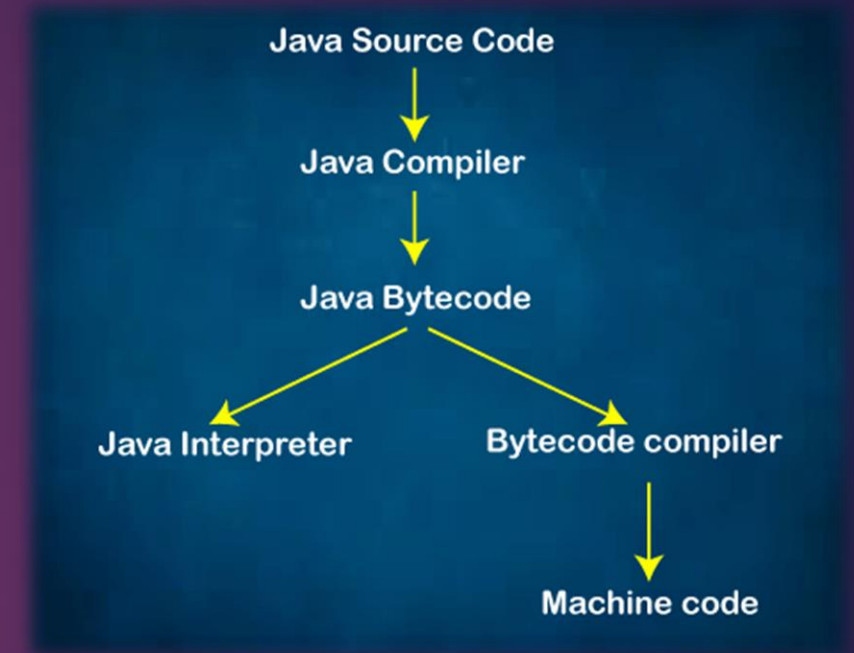
for example :

```
class Basic
{
public static void main(String args[])
{
    System.out.println(" Hello ");
}
}
```

➤ We can execute a Java application by following two steps.

- Compile the Java program
- Execute the application

1. Write Java code and save the file with **.java**
2. Now, this file will be compiled using the Java compiler, which is **javac**.
3. The Java Compiler will compile the Java file and create a **.class** file having byte code (which is not actually a machine code, unlike the C compiler)
4. This generated byte code is a non-executable code, and now it needs an interpreter to convert it into machine code. Here the JVM handles it.
5. Now, JVM will execute this byte code to execute Java byte code on a machine.
6. Now, our program will perform the functionality and gives the desired output.



5] How java is different from java ,C and C++ [8m] [2021,2019,2018,2017,2016,2015,2014,2011]

C	JAVA
It is procedural Language	It is Object Oriented Language
It is compiled	It is interpreted
Error crashes in C	Exception Handling in JAVA
C supports preprocessors	It does not support preprocessors.
It does not support Overloading	It supports overloading
User based Memory Management	Memory Management
It uses pointers	No use of pointers
Top down approach	Bottom Up approach
It is low level Language	It is high level language
Default members are public	Default members are private
It is not portable	It is portable
It is platform dependent	It is platform independent
It supports structure and union	It does not support structure and union
Manual object management	It has garbage collector
It supports go to statement	It does not supports go to statement
It does not support threads	It support threads
It supports multiple inheritance	It does not support multiple inheritance
It supports call by value and call by reference	It supports only call by value
It supports virtual keyword	It does not support virtual keyword
For generic type we use void *	For generic we use Object
Data hiding using static	Data hiding using private
Allocating memory using malloc	Allocating memory using new
It has 32 keywords	It has 50 packages
It supports storage classes	It does not support storage classes

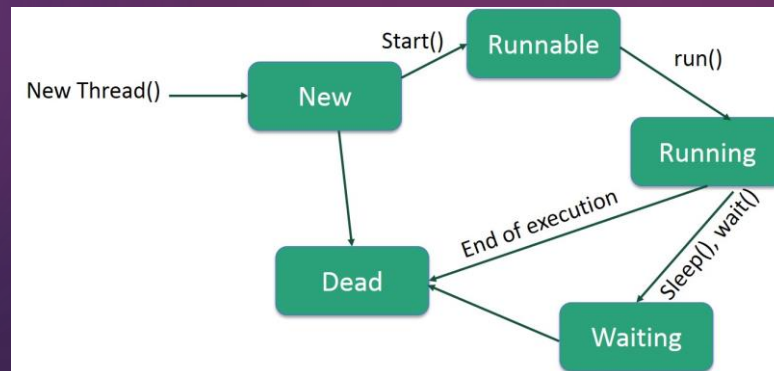
C + +	VERSUS	J A V A
C++		JAVA
A general-purpose programming language that has imperative, object-oriented and generic programming features, while also providing facilities for low-level memory manipulation		A general-purpose computer programming language that is concurrent, class-based, object-oriented, and specifically designed to have as few implementation dependencies as possible
Developed by Bjarne Stroustrup		Developed by Sun Microsystems, initiated by James Gosling
Platform dependent		Platform independent
Supports multiple inheritance		Does not support multiple inheritance directly, but programmer can use interfaces to accomplish this
Compiler-based language as the compiler converts the source code into machine code		Uses both compiler and interpreter as the compiler converts the source code to bytecode and the interpreter executes the bytecode at runtime
Has a goto statement		No goto statement
Supports operator overloading		Does not support operator overloading
Supports structures and unions		Does not support structures and unions
Programmer handles memory management		Makes the system handle memory
Source code is not portable		Bytecode is portable to platform-specific JVMs
More complex		Less complex
Not easy to learn		Easier to learn

6] what is multithread and explain its life cycle . [8m] [2019 ,2018,2011]

- Java is a *multi-threaded programming language* which means we can develop multi-threaded program using Java.
- A multi-threaded program contains two or more parts that can run concurrently and each part can handle a different task at the same time making optimal use of the available resources specially when your computer has multiple CPUs.
- Multi-threading extends the idea of multitasking into applications where you can subdivide specific operations within a single application into individual threads.
- Each of the threads can run in parallel.
- The OS divides processing time not only among different applications, but also among each thread within an application.

Life Cycle of a Thread

- A thread goes through various stages in its life cycle. For example, a thread is born, started, runs, and then dies.



Following are the stages of the life cycle –

- **New** – A new thread begins its life cycle in the new state. It remains in this state until the program starts the thread. It is also referred to as a **born thread**.
- **Runnable** – After a newly born thread is started, the thread becomes runnable. A thread in this state is considered to be executing its task.
- **Waiting** – Sometimes, a thread transitions to the waiting state while the thread waits for another thread to perform a task. A thread transitions back to the runnable state only when another thread signals the waiting thread to continue executing.
- **Timed Waiting** – A runnable thread can enter the timed waiting state for a specified interval of time. A thread in this state transitions back to the runnable state when that time interval expires or when the event it is waiting for occurs.
- **Terminated (Dead)** – A runnable thread enters the terminated state when it completes its task or otherwise terminates.

7] list and explain java API package . [8m] [2019 ,2018,2015,2011]

- The java Standard Library includes hundreds of classes and methods grouped in to several functional packages .
- Most commonly used packages are :
- Language Support Package : A collection of classes and methods required for implementing basic basic features of java.
- Utilities Package : A collection of classes to provide utility functions such as date and time functions.
- Input/output Package : A collection of classes required for input/output manipulation.
- Networking package : A collection of classes for communicating with other computers via internet .
- AWT package : the abstract window tool kit package contains classes that implements platform –independent graphical users interface.
- Applet package : this includes a set of classes that allows us to create java applets.

8] what is an array ? Write the steps to create an array. [6marks]

[2021, 2018]

Java Arrays

Normally, an array is a collection of similar type of elements which has contiguous memory location.

How to Create Array of Objects in Java

Array of Objects in Java

- Java is an object-oriented programming language.
- Most of the work done with the help of **objects**.
- We know that an array is a collection of the same data type that dynamically creates objects and can have elements of primitive types. Java allows us to store objects in an array.
- In Java the class is also a user-defined data type.
- An array that contains **class type elements** are known as an **array of objects**.
- It stores the reference variable of the object.

Creating an Array of Objects

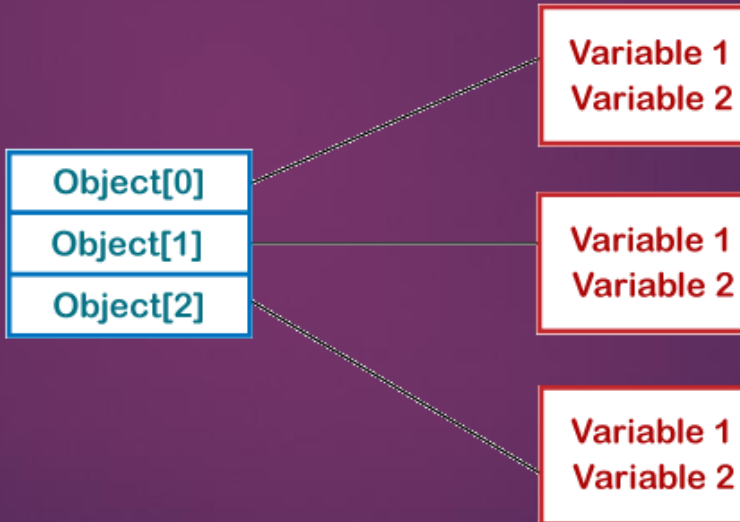
- Before creating an array of objects, we must create an instance of the class by using the new keyword.
- We can use any of the following statements to create an array of objects.

Syntax:

```
ClassName[] objArray;
```

Or

```
ClassName objeArray[];
```



9] what is narrowing conversion? [2marks]

[2021,2015,2014]

- Narrowing conversion is needed when you convert from a larger size type to a smaller size.
- This is for incompatible data types, wherein automatic conversions cannot be done.

➤ Let us see an example wherein we are converting long to integer using Narrowing Conversion.

Example

```
public class Demo {  
    public static void main(String[] args) {  
  
        long longVal = 878;  
        int intVal = (int) longVal;  
        System.out.println("Long: "+longVal);  
        System.out.println("Integer: "+intVal);  
    }  
}
```

Output

```
Long: 878  
Integer: 878
```

10] what are the types of inheritance? Explain them briefly. [8marks]

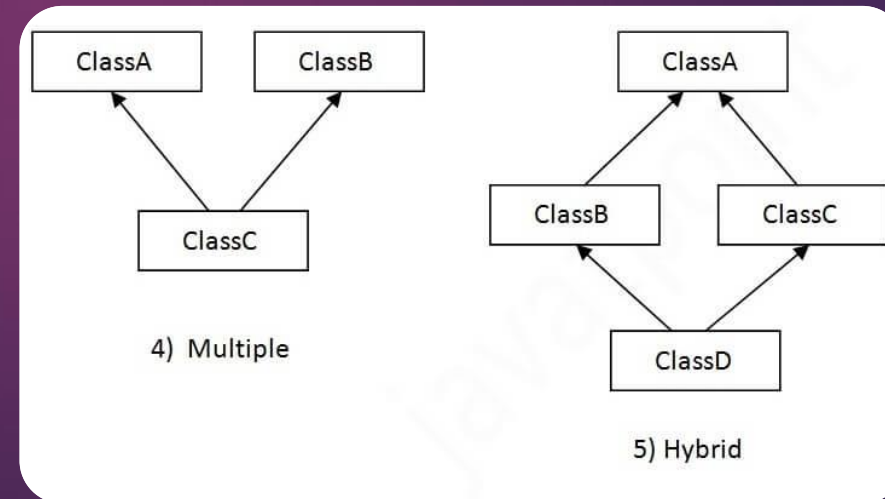
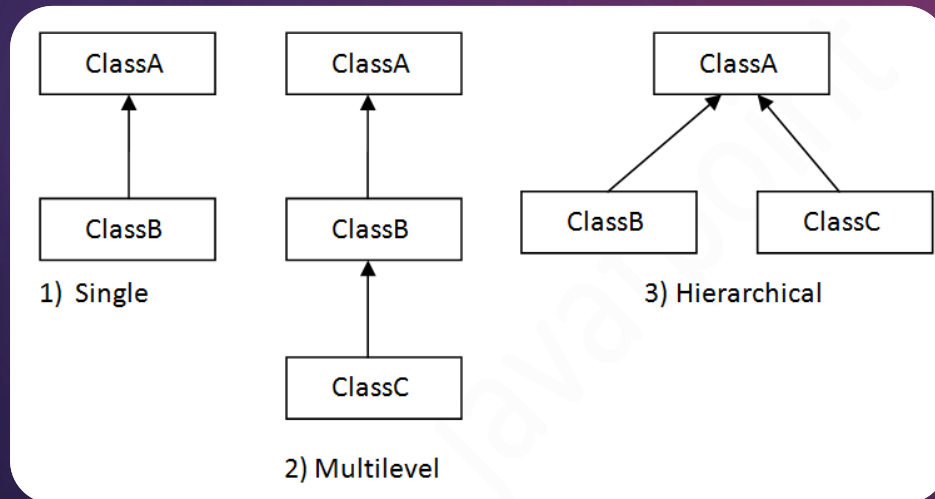
[2021, 2019 ,2017, 2016, 2015,2011]

Inheritance in Java

- **Inheritance in Java** is a mechanism in which one object acquires all the properties and behaviors of a parent object. It is an important part of OOPs.
- (Object Oriented programming system).

Types of inheritance in java

- On the basis of class, there can be three types of inheritance in java: single, multilevel and hierarchical.
- Multiple inheritance is not supported in Java through class.



- When one class inherits multiple classes, it is known as multiple inheritance.
For Example:

✓ Single Inheritance

- When a class inherits another class, it is known as a single inheritance.

Single Inheritance Example

In the example given below, Dog class inherits the Animal class, so there is the single inheritance.

```
class Anima
{
void eat(){System.out.println("eating...");}
}
class Dog extends Anima{
void bark(){System.out.println("barking...");}
}
class TestInheritance{
public static void main(String args[]){
    Dog d=new Dog();
    d.bark();
    d.eat();
}
}
```

Output:

barking...
eating...

✓ Multilevel Inheritance

- When there is a chain of inheritance, it is known as multilevel inheritance
- The example given below, BabyDog class inherits the Dog class which again inherits the Animal class, so there is a multilevel inheritance.

```
class Animal{
void eat(){System.out.println("eating...");}
}
class Dog extends Animal{
void bark(){System.out.println("barking...");}
}
class BabyDog extends Dog{
void weep(){System.out.println("weeping...");}
}
class TestInheritance2{
public static void main(String args[]){
BabyDog d=new BabyDog();
d.weep();
d.bark();
d.eat();
}}
```

Output:

weeping...
barking...
eating...

✓ Hierarchical Inheritance

- When two or more classes inherits a single class, it is known as hierarchical inheritance
- In the example given below, Dog and Cat classes inherits the Animal class, so there is hierarchical inheritance.

Hierarchical Inheritance Example

```
class Animal{  
void eat(){System.out.println("eating...");}  
}  
class Dog extends Animal{  
void bark(){System.out.println("barking...");}  
}  
class Cat extends Animal{  
void meow(){System.out.println("meowing...");}  
}  
class TestInheritance3{  
public static void main(String args[]){  
    Cat c=new Cat();  
    c.meow();  
    c.eat();  
    //c.bark();//C.T.Error  
}}
```

Output:

meowing...
eating...

❑ Why multiple inheritance is not supported in java?

To reduce the complexity and simplify the language, multiple inheritance is not supported in java.

11] How to Handle exceptions and program to demonstrate finally keyword?. [8marks] [2021, 2018, 2015]

Exception Handling in Java

- The **Exception Handling in Java** is one of the powerful *mechanism to handle the runtime errors* so that the normal flow of the application can be maintained.
- In Java, an exception is an event that disrupts the normal flow of the program. It is an object which is thrown at runtime.

Advantage of Exception Handling

- The core advantage of exception handling is **to maintain the normal flow of the application.**
- An exception normally disrupts the normal flow of the application; that is why we need to handle exceptions.

finally keyword

- Final(lowercase) is a reserved keyword in java.
- We can't use it as an identifier as it is reserved.
- We can use this keyword with variables, methods and also with classes.
- The final keyword in java has different meaning depending upon it is applied to variable, class or method.
- Final with Variables : The value of variable cannot be changed once initialized.

```
class A {  
    public static void main(String[] args)  
    {  
        // Non final variable  
        int a = 5;  
  
        // final variable  
        final int b = 6;  
  
        // modifying the non final variable : Allowed  
        a++;  
  
        // modifying the final variable :  
        // Immediately gives Compile Time error.  
        b++;  
    }  
}
```

- If we declare any variable as final, we can't modify its contents since it is final, and if we modify it then we get Compile Time Error.

12] what is package ? Write a program to demonstrate user package [8marks] [2021, 2019,2017,2016,2015,2014,2011]

Java Package

- A java package is a group of similar types of classes, interfaces and sub-packages.
- Package in java can be categorized in two form, built-in package and user-defined package.
- There are many built-in packages such as java, lang, awt, javax, swing, net, io, util, sql etc.

Advantage of Java Package

- 1) Java package is used to categorize the classes and interfaces so that they can be easily maintained.
- 2) Java package provides access protection.
- 3) Java package removes naming collision.

A program to demonstrate user package

User-defined packages

These are the packages that are defined by the user. First we create a directory my Package (name should be same as the name of the package). Then create the My Class inside the directory with the first statement being the package names.

```
// Name of the package must be same as the directory
// under which this file is saved
package myPackage;

public class MyClass
{
    public void getNames(String s)
    {
        System.out.println(s);
    }
}
```

13] what is thread ? Explain life cycle. [8marks]

[2021, 2017, 2016, 2014,2011]

Thread

A **Thread** is a very light-weighted process, or we can say the smallest part of the process that allows a program to operate more efficiently by running multiple tasks simultaneously.

1) New (Ready to run)

A thread is in **New** when it gets CPU time.

2) Running

A thread is in a **Running** state when it is under execution.

3) Suspended

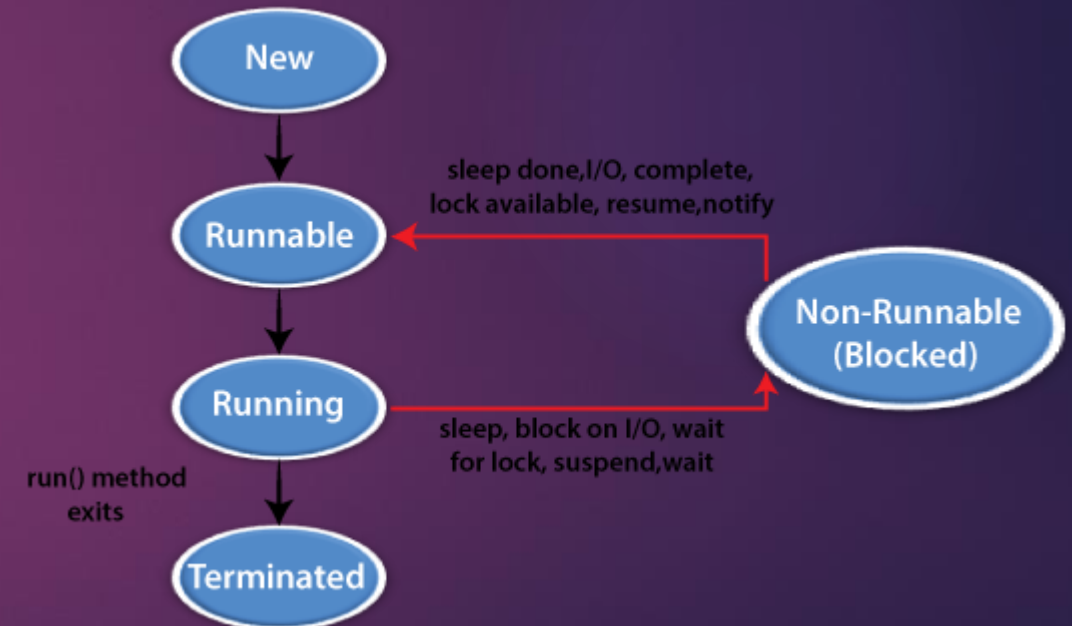
A thread is in the **Suspended** state when it is temporarily inactive or under execution.

4) Blocked

A thread is in the **Blocked** state when it is waiting for resources.

5) Terminated

A thread comes in this state when at any given time, it halts its execution immediately.

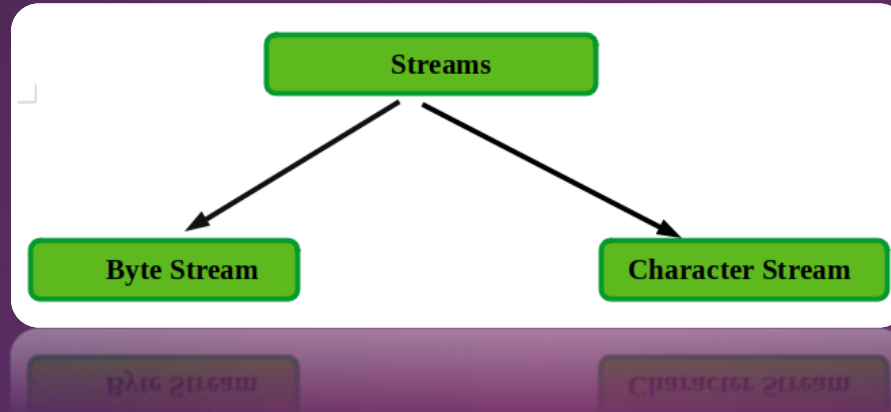


14] Explain briefly the input stream class and output stream class. [8marks]

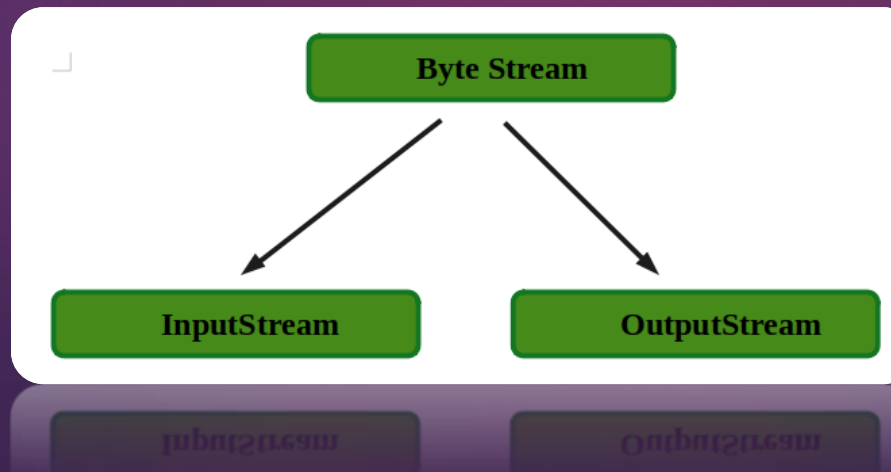
[2021,2018,2017,2015,2011,2014]

stream

- A **stream** can be defined as the sequence of data or continuous flow of data. Streams are a clear way to deal with Input/Output.

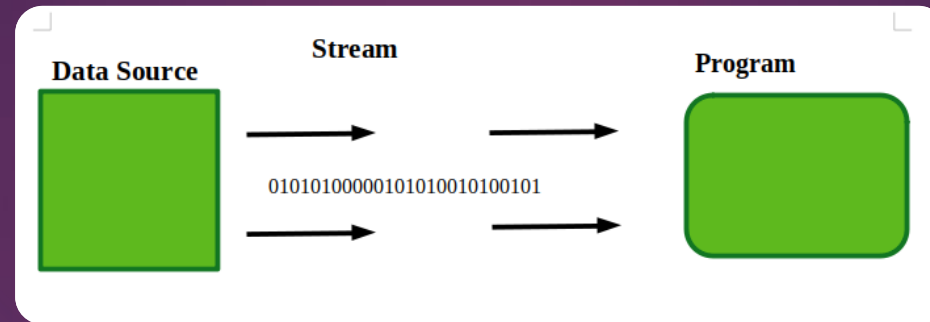


- Byte Stream:** Byte Stream provides a convenient way of handling the input and output of byte. The byte stream is further divided into various classes but the top hierarchy Classes are depicted below:



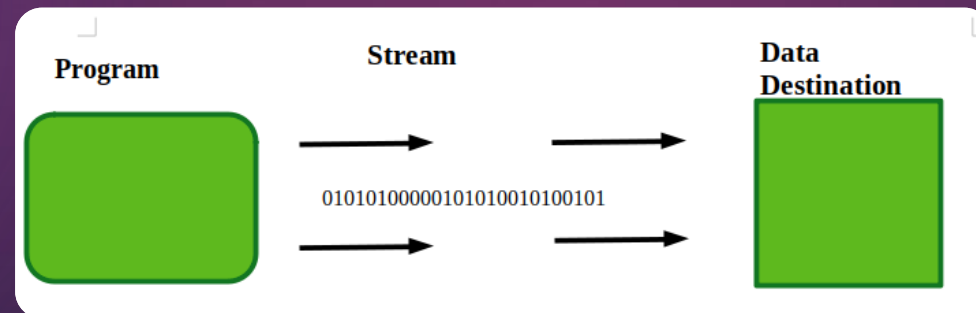
Input Stream :

Input Stream is an abstract class of Byte Stream that describe stream input and it is used for reading and it could be a file, image, audio, video, webpage, etc. it doesn't matter. Thus, Input Stream read data from source one item at a time.

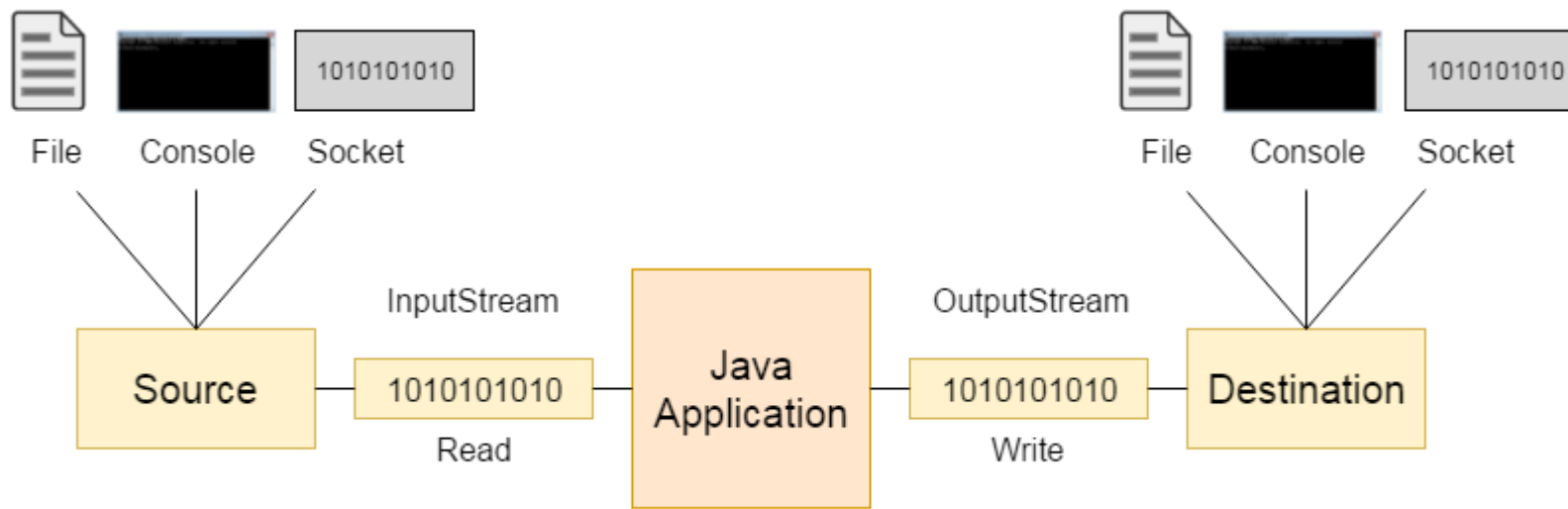


Output Stream:

Output Stream is an abstract class of Byte Stream that describes stream output and it is used for writing data to a file, image, audio, etc. Thus, Output Stream writes data to the destination one at a time.



input stream class and output stream class.

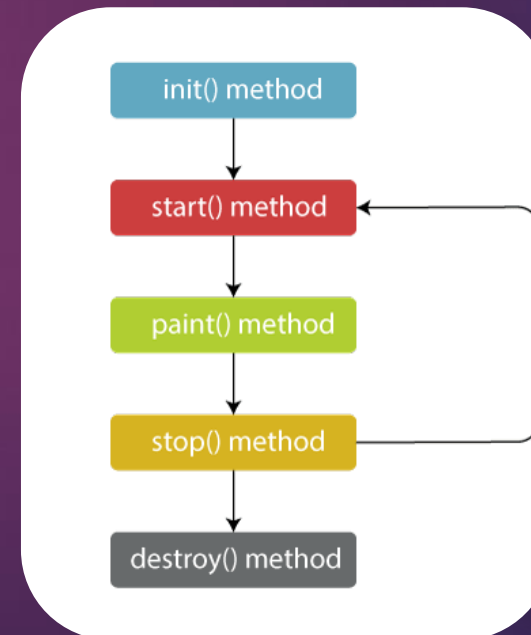


15] Define Applet and explain its life cycle. [8marks]

[2021, 2017, 2018, 2016, 2015, 2014, 2011]

Applet Life Cycle in Java

- The Java plug-in software is responsible for managing the life cycle of an applet.
 - An applet is a Java application executed in any web browser and works on the client-side. It doesn't have the main() method because it runs in the browser. It is thus created to be placed on an HTML page.
 - The init(), start(), stop() and destroy() methods belongs to the **applet. Applet** class.
 - The paint() method belongs to the **awt. Component** class.
 - In Java, if we want to make a class an Applet class, we need to extend the **Applet**
 - Whenever we create an applet, we are creating the instance of the existing Applet class. And thus, we can use all the methods of that class.
-
- These methods are invoked by the browser automatically. There is no need to call them explicitly.

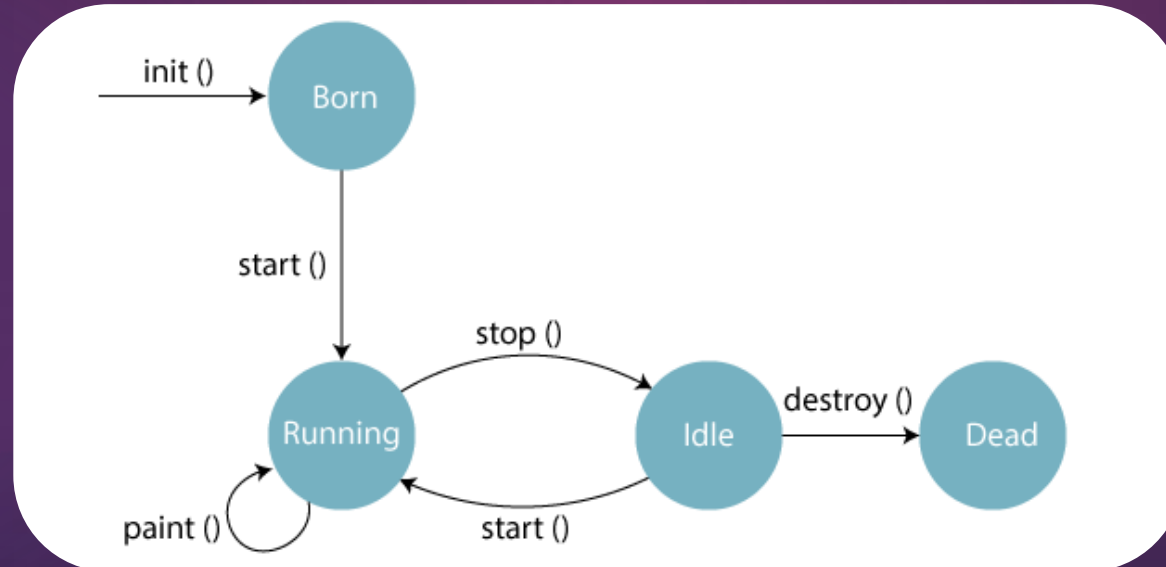


Syntax of entire Applet Life Cycle in Java

```
class TestAppletLifeCycle extends Applet {  
public void init() {  
    // initialized objects  
}  
public void start() {  
    // code to start the applet  
}  
public void paint(Graphics graphics) {  
    // draw the shapes  
}  
public void stop() {  
    // code to stop the applet  
}  
public void destroy() {  
    // code to destroy the applet  
}  
}
```

Methods of Applet Life Cycle

- In Java, an applet is a special type of program embedded in the web page to generate dynamic content. Applet is a class in Java.
- The applet life cycle can be defined as the process of how the object is created, started, stopped, and destroyed during the entire execution of its application. It basically has five core methods namely `init()`, `start()`, `stop()`, `paint()` and `destroy()`. These methods are invoked by the browser to execute.



- **init():**

The init() method is the first method to run that initializes the applet. It can be invoked only once at the time of initialization. The web browser creates the initialized objects, i.e., the web browser (after checking the security settings) runs the init() method within the applet.

- **start():**

The start() method contains the actual code of the applet and starts the applet. It is invoked immediately after the init() method is invoked. Every time the browser is loaded or refreshed, the start() method is invoked. It is also invoked whenever the applet is maximized, restored, or moving from one tab to another in the browser. It is in an inactive state until the init() method is invoked.

- **stop():**

The stop() method stops the execution of the applet. The stop () method is invoked whenever the applet is stopped, minimized, or moving from one tab to another in the browser, the stop() method is invoked. When we go back to that page, the start() method is invoked again.

- **destroy():**

The destroy() method destroys the applet after its work is done. It is invoked when the applet window is closed or when the tab containing the webpage is closed. It removes the applet object from memory and is executed only once. We cannot start the applet once it is destroyed.

- **paint():**

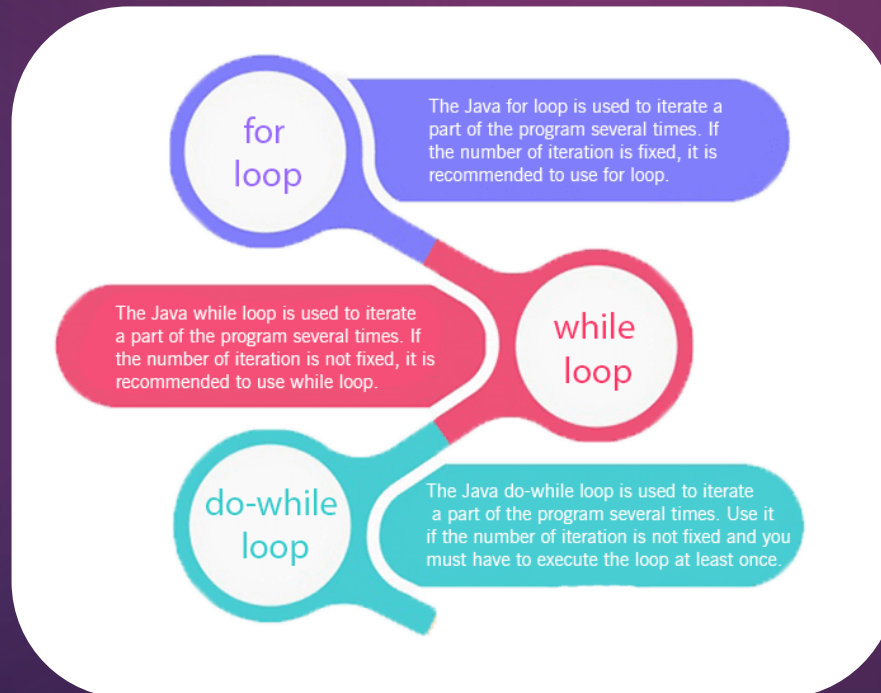
The paint() method belongs to the Graphics class in Java. It is used to draw shapes like circle, square, trapezium, etc., in the applet. It is executed after the start() method and when the browser or applet windows are resized.

16] Briefly explain the looping statements [8marks] [2021,2018,2017,2016, 2015, 2014, 2011]

Loops in Java

The Java *for loop* is used to iterate a part of the program several times. If the number of iteration is **fixed**, it is recommended to use for loop.

There are three types of for loops in Java.



Java Simple for Loop

- A simple for loop is the same as C/C++
- We can initialize the variable
- check condition and increment/decrement value. It consists of four parts:

- **Initialization** :

It is the initial condition which is executed once when the loop starts. Here, we can initialize the variable, or we can use an already initialized variable. It is an optional condition.

- **Condition** :

It is the second condition which is executed each time to test the condition of the loop. It continues execution until the condition is false. It must return Boolean value either true or false. It is an optional condition.

- **Increment/Decrement** :

It increments or decrements the variable value. It is an optional condition.

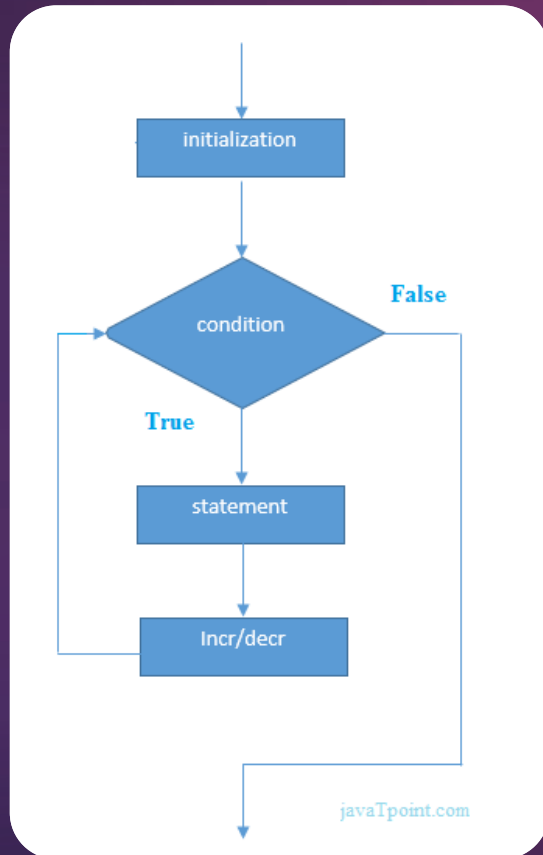
- **Statement** :

The statement of the loop is executed each time until the second condition is false.

Syntax :

```
for(initialization; condition; increment/decrement)
{
//statement or code to be executed
}
```

Flowchart:



Example :

//Java Program to demonstrate the example of for loop
//which prints table of 1

```
public class ForExample
{
public static void main(String[] args)
{
//Code of Java for loop
for(int i=1;i<=10;i++)
{
    System.out.println(i);
}
}
```

Output:

1
2
3
4
5
6
7
8
9
10

Bca expert

Java for Loop vs while Loop vs do-while Loop

Comparison	for loop	while loop	do-while loop
Introduction	The Java for loop is a control flow statement that iterates a part of the programs multiple times.	The Java while loop is a control flow statement that executes a part of the programs repeatedly on the basis of given Boolean condition.	The Java do while loop is a control flow statement that executes a part of the programs at least once and the further execution depends upon the given Boolean condition.
When to use	If the number of iteration is fixed, it is recommended to use for loop.	If the number of iteration is not fixed, it is recommended to use while loop.	If the number of iteration is not fixed and you must have to execute the loop at least once, it is recommended to use the do-while loop.
Syntax	<pre>for(init;condition;incr/de cr){ // code to be executed }</pre>	<pre>while(condition){ //code to be executed }</pre>	<pre>do{ //code to be executed }while(condition);</pre>

Comparison	for loop	while loop	do-while loop
Example	<pre>//for loop for(int i=1;i<=10;i++){ System.out.println(i); }</pre>	<pre>//while loop int i=1; while(i<=10){ System.out.println(i); i++; }</pre>	<pre>//do-while loop int i=1; do{ System.out.println(i); i++; }while(i<=10);</pre>
Syntax for infinitive loop	<pre>for(;;){ //code to be executed }</pre>	<pre>while(true){ //code to be executed }</pre>	<pre>do{ //code to be executed }while(true);</pre>

17] what is overloading [6marks] [2021, 2018, 2016,2011]

Overloading in Java

- If a class has multiple methods having same name but different in parameters, it is known as Method Overloading.
- If we have to perform only one operation, having same name of the methods increases the readability of the program.
- The overloading is also known as method overloading.
- Suppose you have to perform addition of the given numbers but there can be any number of arguments, if you write the method such as a(int,int) for two parameters, and b(int,int,int) for three parameters then it may be difficult for you as well as other programmers to understand the behavior of the method because its name differs.

Advantage of method overloading

- Method overloading *increases the readability of the program.*

Different ways to overload the method

There are two ways to overload the method in java

1. By changing number of arguments
2. By changing the data type

Difference between Vector & Array

Sr. No.	Vector	Array
1	Vector can grow and shrink dynamically.	Array can't grow and shrink dynamically.
2	Vector can hold dynamic list of objects or primitive data types.	Array is a static list of primitive data types.
3	Vector class is found in java.util package .	Array class is found in java.lang (default) package.
4	Vector can store elements of different data types.	Array can store elements of same data type.
5	Vector class provides different methods for accessing and managing vector elements.	For accessing element of an array no special methods are available as it is not a class, but a derived type.
6	Syntax : Vector objectname=new Vector();	Syntax : datatype arrayname[]=new datatype[size];
7	Example: Vector v1=new Vector();	Example: int[] myArray={22,12,9,44};

Priority of a Thread (Thread Priority)

- Each thread has a priority. Priorities are represented by a number between 1 and 10.
- In most cases, the thread scheduler schedules the threads according to their priority (known as preemptive scheduling). But it is not guaranteed because it depends on JVM specification that which scheduling it chooses.
- Note that not only JVM a Java programmer can also assign the priorities of a thread explicitly in a Java program.

Setter & Getter Method of Thread Priority

public final int getPriority() :

The `java.lang.Thread.getPriority()` method returns the priority of the given thread.

public final void setPriority(int newPriority) :

The `java.lang.Thread.setPriority()` method updates or assign the priority of the thread to `newPriority`. The method throws `IllegalArgumentException` if the value `newPriority` goes out of the range, which is 1 (minimum) to 10 (maximum).

3 constants defined in Thread class :

- `public static int MIN_PRIORITY`
- `public static int NORM_PRIORITY`
- `public static int MAX_PRIORITY`

➤ Default priority of a thread is 5 (NORM_PRIORITY). The value of MIN_PRIORITY is 1 and the value of MAX_PRIORITY is 10.

20] How to pass parameters from html in applet ?explain with example [8marks] [2021]

Parameter in Applet

We can get any information from the HTML file as a parameter. For this purpose, Applet class provides a method named `getParameter()`.

Syntax :

```
public String getParameter(String parameterName)
```

Example of using parameter in Applet:

myapplet.html

```
import java.applet.Applet;
import java.awt.Graphics;

public class UseParam extends Applet
{
    public void paint(Graphics g)
    {
        String str=getParameter("msg");
        g.drawString(str,50, 50);
    }
}
```

```
<html>
<body>
<applet code="UseParam.class" width="300" height="300">
<param name="msg" value="Welcome to applet">
</applet>
</body>
</html>
```

21] what is tokens? Explain its types [8marks]

[2019, 2018]

Java Tokens

- In Java, the program contains classes and methods.
- Further, the methods contain the expressions and statements required to perform a specific operation. These statements and expressions are made up of **tokens**.
- In other words, we can say that the expression and statement is a set of **tokens**.
- The tokens are the small building blocks of a Java program that are meaningful to the Java compiler. Further, these two components contain variables, constants, and operators.
- In this section, we will discuss **what is tokens in Java**.

What is token in Java?

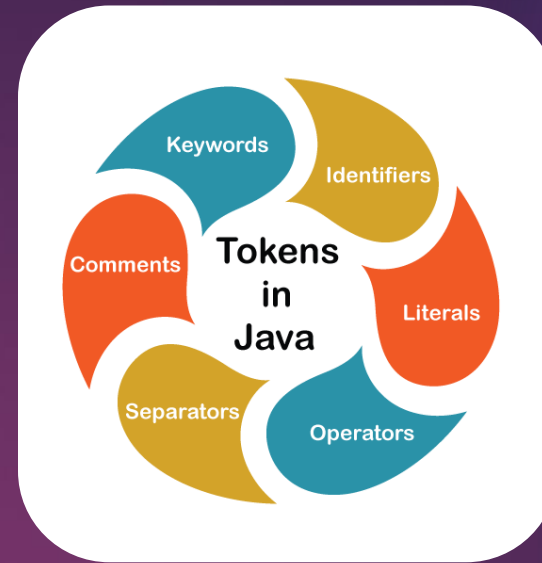
- The Java compiler breaks the line of code into text (words) is called **Java tokens**.
- These are the smallest element of the Java program.
- The Java compiler identified these words as tokens. These tokens are separated by the delimiters.
- It is useful for compilers to detect errors.
- Remember that the delimiters are not part of the Java tokens.

➤ token <= identifier | keyword | separator | operator | literal | comment

Types of Tokens

Java token includes the following :

1. Keywords
2. Identifiers
3. Literals
4. Operators
5. Separators
6. Comments



- **Keywords** : These are the **pre-defined** reserved words of any programming language. Each keyword has a special meaning. It is always written in lower case.
- **Identifier** : Identifiers are used to name a variable, constant, function, class, and array. It usually defined by the user.
- **Literals** : In programming literal is a notation that represents a fixed value (constant) in the source code. It can be categorized as an integer literal, string literal, Boolean literal, etc. It is defined by the programmer
- **Operators** : In programming, operators are the special symbol that tells the compiler to perform a special operation. Java provides different types of operators that can be classified according to the functionality they provide.
- **Separators** : The separators in Java is also known as **punctuators**.
- **Comments** : Comments allow us to specify information about the program inside our Java code. Java compiler recognizes these comments as tokens but excludes it from further processing

22] what is constant ? How to make a variable value as a constant ? Explain with example [8 marks] [2019,2014]

Java Constant

Constant is a value that cannot be changed after assigning it. Java does not directly support the constants. There is an alternative way to define the constants in Java by using the non-access modifiers static and final. How to declare constant in Java?

How to declare constant in Java?

In Java, to declare any variable as constant, we use static and final modifiers. It is also known as **non-access** modifiers. According to the Java naming convention the identifier name must be in **capital letters**.

Static and Final Modifiers

- The purpose to use the static modifier is to manage the memory.
- It also allows the variable to be available without loading any instance of the class in which it is defined.
- The final modifier represents that the value of the variable cannot be changed. It also makes the primitive data type immutable or unchangeable.

The syntax to declare a constant

1. **static final** datatype identifier name = value;

Declaring Constant as Private

```
import java.util.Scanner;
public class ConstantExample1
{
//declaring constant
private static final double PRICE=234.90;
public static void main(String[] args)
{
int unit;
double total_bill;
System.out.print("Enter the number of units you have used: ");
Scanner sc =new Scanner(System.in);
Unit = sc.nextInt();
total_bill=PRICE*unit;
System.out.println("The total amount you have to deposit is: "+total_bill);
}
}
```

Output

```
Enter the number of units you have used: 10
The total amount you have to deposit is: 2349.0
```

23] How to make use of bitwise operators in java program ? Explain with example [8 marks] [2019,2015]

Bitwise Operator in Java

In Java, an **operator** is a symbol that performs the specified operations. In this section, we will discuss only the **bitwise operator and its types** with proper examples.

Types of Bitwise Operator

There are six types of the bitwise operator in Java:

- Bitwise AND
- Bitwise exclusive OR
- Bitwise inclusive OR
- Bitwise Compliment
- Bit Shift Operators

Bitwise AND (&)

It is a binary operator denoted by the symbol &. It returns 1 if and only if both bits are 1, else returns 0.

x	y	x & y
0	0	0
0	1	0
1	0	0
1	1	1

BitwiseAndExample.java

```
public class BitwiseAndExample
{
    public static void main(String[] args)
    {
        int x = 9, y = 8;
        // bitwise and
        // 1001 & 1000 = 1000 = 8
        System.out.println("x & y = " + (x & y));
    }
}
```

Output

x & y = 8

Bitwise inclusive OR (|)

It is a binary operator denoted by the symbol | (pronounced as a pipe). It returns 1 if either of the bit is 1, else returns 0.

x	y	x y
0	0	0
0	1	1
1	0	1
1	1	1

Bitwise Inclusive Or Example.java

```
public class BitwiseInclusiveOrExample
{
    public static void main(String[] args)
    {
        int x = 9, y = 8;
        // bitwise inclusive OR
        // 1001 | 1000 = 1001 = 9
        System.out.println("x | y = " + (x | y));
    }
}
```

Output

x | y = 9

Bitwise Complement (~)

It is a unary operator denoted by the symbol ~ (pronounced as the tilde). It returns the inverse or complement of the bit. It makes every 0 a 1 and every 1 a 0.

X	~X
0	1
1	0

BitwiseComplimentExample.java

```
public class BitwiseComplimentExample
{
    public static void main(String[] args)
    {
        int x = 2;
        // bitwise compliment
        // ~0010= 1101 = -3
        System.out.println("~x = " + (~x));
    }
}
```

Output

~x = -3

24] Define class ? Explain visibility modes with example

[8 marks]

[2019,2017,2015]

A class is a group of objects which have common properties. It is a template or blueprint from which objects are created. It is a logical entity.

visibility modes

The access modifiers in Java specifies the accessibility or scope of a field, method, constructor, or class. We can change the access level of fields, constructors, methods, and class by applying the access modifier on it.

1.Private : The access level of a private modifier is only within the class. It cannot be accessed from outside the class.

2.Default : The access level of a default modifier is only within the package. It cannot be accessed from outside the package. If you do not specify any access level, it will be the default.

3.Protected : The access level of a protected modifier is within the package and outside the package through child class. If you do not make the child class, it cannot be accessed from outside the package.

4.Public : The access level of a public modifier is everywhere. It can be accessed from within the class, outside the class, within the package and outside the package.

Access Modifier	within class	within package	outside package by subclass only	outside package
Private	Y	N	N	N
Default	Y	Y	N	N
Protected	Y	Y	Y	N
Public	Y	Y	Y	Y

1) Private

The private access modifier is accessible only within the class.

Simple example of private access modifier

In this example, we have created two classes A and Simple. A class contains private data member and private method. We are accessing these private members from outside the class, so there is a compile-time error.

```
class A{  
    private int data=40;  
    private void msg(){System.out.println("Hello java");}  
}
```

```
public class Simple{  
    public static void main(String args[]){  
        A obj=new A();  
        System.out.println(obj.data);//Compile Time Error  
        obj.msg();//Compile Time Error  
    }  
}
```

2) Default

If you don't use any modifier, it is treated as **default** by default. The default modifier is accessible only within package. It cannot be accessed from outside the package. It provides more accessibility than private. But, it is more restrictive than protected, and public.

Example of default access modifier

In this example, we have created two packages pack and my pack. We are accessing the A class from outside its package, since A class is not public, so it cannot be accessed from outside the package.

```
//save by A.java
package pack;
class A{
    void msg(){System.out.println("Hello");}
}
```

```
//save by B.java
package mypack;
import pack.*;
class B{
    public static void main(String args[]){
        A obj = new A();//Compile Time Error
        obj.msg();//Compile Time Error
    }
}
```

- In this example, the scope of class A and its method msg() is default so it cannot be accessed from outside the package.

3) Protected

The **protected access modifier** is accessible within package and outside the package but through inheritance only.

The protected access modifier can be applied on the data member, method and constructor. It can't be applied on the class.

It provides more accessibility than the default modifier.

Example of protected access modifier

In this example, we have created the two packages pack and mypack. The A class of pack package is public, so can be accessed from outside the package. But msg method of this package is declared as protected, so it can be accessed from outside the class only through inheritance.

```
//save by A.java
package pack;
public class A{
protected void msg(){System.out.println("Hello");}
}

//save by B.java
package mypack;
import pack.*;
class B extends A{
public static void main(String args[]){
    B obj = new B();
    obj.msg();
}
}
```

4) Public

The public access modifier is accessible everywhere. It has the widest scope among all other modifiers.

Example of public access modifier

```
//save by A.java
```

```
package pack;  
public class A{  
  public void msg(){System.out.println("Hello");}  
}
```

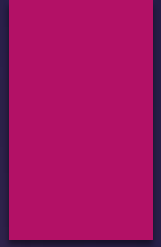
```
//save by B.java
```

```
package mypack;  
import pack.*;  
  
class B{  
  public static void main(String args[]){  
    A obj = new A();  
    obj.msg();  
  }  
}
```

Output : Hello

25] Explain any 4 mathematical methods available in java with syntax and example.
[8 marks] [2019,2014]

java Math class provides several methods to work on math calculations like min(), max(), avg(), sin(), cos(), tan(), round(), ceil(), floor(), abs() etc.



Method Overriding in Java

If subclass (child class) has the same method as declared in the parent class, it is known as **method overriding in Java**.

In other words, If a subclass provides the specific implementation of the method that has been declared by one of its parent class, it is known as method overriding.

Usage of Java Method Overriding

- Method overriding is used to provide the specific implementation of a method which is already provided by its superclass.
- Method overriding is used for runtime polymorphism

Rules for Java Method Overriding

- The method must have the same name as in the parent class
- The method must have the same parameter as in the parent class.
- There must be an IS-A relationship (inheritance).

27] what is vector class? What are the advantages and disadvantages of vector class method. [8 marks]

[2019,2015,2014]

Vector is like the dynamic array which can grow or shrink its size. Unlike array, we can store n-number of elements in it as there is no size limit. It is recommended to use the Vector class in the thread-safe implementation only.

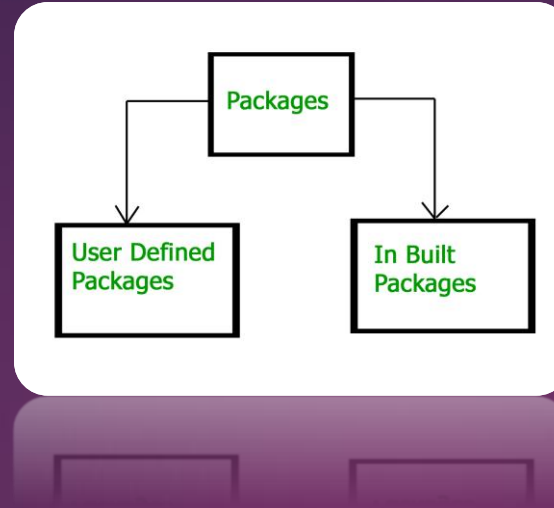
Advantages

- The big advantage of using Vectors is that the size of the vector can change as needed.
- Vectors handle these changes through the "capacity" and "capacity Increment" fields.
- When a Vector is instantiated, it declares an object array of size initial Capacity.
- Whenever this array fills, the vector increases the total buffer size by the value capacity Increment.
- Thus, a Vector represents a compromise.
- It doesn't allocate each object dynamically, which would make access of middle list-items exceedingly slow; but it does allow for growth in the size of the list.
- The default constructor, Vector(), creates an empty Vector of initial capacity zero that doubles in size whenever it fills.

Disadvantages

- A vector is an object, memory consumption is more.

**28] Discuss different type of system package and user defined packages. [8 marks]
[2019,2015,2014]**



Built-in Packages(system package)

These packages consist of a large number of classes which are a part of Java **API**. Some of the commonly used built-in packages are:

- 1) **java. Lang** : Contains language support classes(e.g classed which defines primitive data types, math operations). This package is automatically imported.
- 2) **java.io** : Contains classed for supporting input / output operations.
- 3) **java.util** : Contains utility classes which implement data structures like Linked List, Dictionary and support ; for Date / Time operations.
- 4) **java. Applet** : Contains classes for creating Applets.
- 5) **java.awt** : Contain classes for implementing the components for graphical user interfaces (like button , ;menus etc).
- 6) **java.net** : Contain classes for supporting networking operations.

User-defined packages

These are the packages that are defined by the user. First we create a directory **myPackage** (name should be same as the name of the package). Then create the **MyClass** inside the directory with the first statement being the **package names**.

- Every class is part of some package.
- If no package is specified, the classes in the file goes into a special unnamed package (the same unnamed package for all files).
- All classes/interfaces in a file are part of the same package. Multiple files can specify the same package name.
- If package name is specified, the file must be in a subdirectory called name (i.e., the directory name must match the package name).
- We can access public classes in another (named) package using: **package-name.class-name**

```
// Name of the package must be same as the directory  
// under which this file is saved package myPackage;
```

```
public class MyClass  
{  
    public void getNames(String s)  
    {  
        System.out.println(s);  
    }  
}
```

30] Compare thread class with runnable interface. [6 marks]

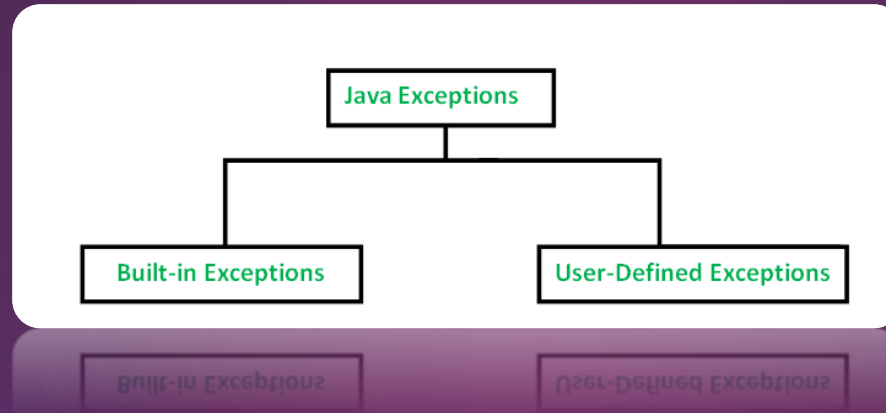
[2019]

Sr. No.	Key	Thread	Runnable
1	Basic	Thread is a class. It is used to create a thread	Runnable is a functional interface which is used to create a thread
2	Methods	It has multiple methods including start() and run()	It has only abstract method run()
3		Each thread creates a unique object and gets associated with it	Multiple threads share the same objects.
4	Memory	More memory required	Less memory required
5	Limitation	Multiple Inheritance is not allowed in java hence after a class extends Thread class, it can not extend any other class	If a class is implementing the runnable interface then your class can extend another class.

31] explain different types of exception and their usages . [10 marks]

[2019,2015]

Java defines several types of exceptions that relate to its various class libraries. Java also allows users to define their own exceptions.



Built-in exceptions

Built-in exceptions are the exceptions which are available in Java libraries. These exceptions are suitable to explain certain error situations. Below is the list of important built-in exceptions in Java.

1.Arithmetic Exception

It is thrown when an exceptional condition has occurred in an arithmetic operation.

2.Array Index Out Of Bounds Exception

It is thrown to indicate that an array has been accessed with an illegal index. The index is either negative or greater than or equal to the size of the array.

3.Class Not Found Exception

This Exception is raised when we try to access a class whose definition is not found

4.File Not Found Exception

This Exception is raised when a file is not accessible or does not open.

5.IO Exception

It is thrown when an input-output operation failed or interrupted

6.Interrupted Exception

It is thrown when a thread is waiting, sleeping, or doing some processing, and it is interrupted.

7.No Such Field Exception

It is thrown when a class does not contain the field (or variable) specified

8.No Such Method Exception

It is thrown when accessing a method which is not found.

9.Null Pointer Exception

This exception is raised when referring to the members of a null object. Null represents nothing

10.Number Format Exception

This exception is raised when a method could not convert a string into a numeric format.

11.Runtime Exception

This represents any exception which occurs during runtime.

Byte Stream Classes in Java

Byte Stream classes are used to read bytes from the input stream and write bytes to the output stream. In other words, we can say that Byte Stream classes read/write the data of 8-bits. We can store video, audio, characters, etc., by using Byte Stream classes. These classes are part of the java.io package.

The Byte Stream classes are divided into two types of classes, i.e., Input Stream and Output Stream. These classes are abstract and the super classes of all the Input/Output stream classes.

Input Stream Class

The Input Stream class provides methods to read bytes from a file, console or memory. It is an abstract class and can't be instantiated; however, various classes inherit the Input Stream class and override its methods.

Output Stream Class

The Output Stream is an abstract class that is used to write 8-bit bytes to the stream. It is the superclass of all the output stream classes. This class can't be instantiated; however, it is inherited by various subclasses that are given in the following table.

character stream class

The java.io package provides Character Stream classes to overcome the limitations of Byte Stream classes, which can only handle the 8-bit bytes and is not compatible to work directly with the Unicode characters. Character Stream classes are used to work with 16-bit Unicode characters. They can perform operations on characters, char arrays and Strings. the Character Stream classes are mainly used to read characters from the source and write them to the destination

Reader Class

Reader class is used to read the 16-bit characters from the input stream. However, it is an abstract class and can't be instantiated, but there are various subclasses that inherit the Reader class and override the methods of the Reader class. All methods of the Reader class throw an IO Exception

Writer Class

Writer class is used to write 16-bit Unicode characters to the output stream. The methods of the Writer class generate IO Exception. Like Reader class, Writer class is also an abstract class that cannot be instantiated; therefore, the subclasses of the Writer class are used to write the characters onto the output stream.

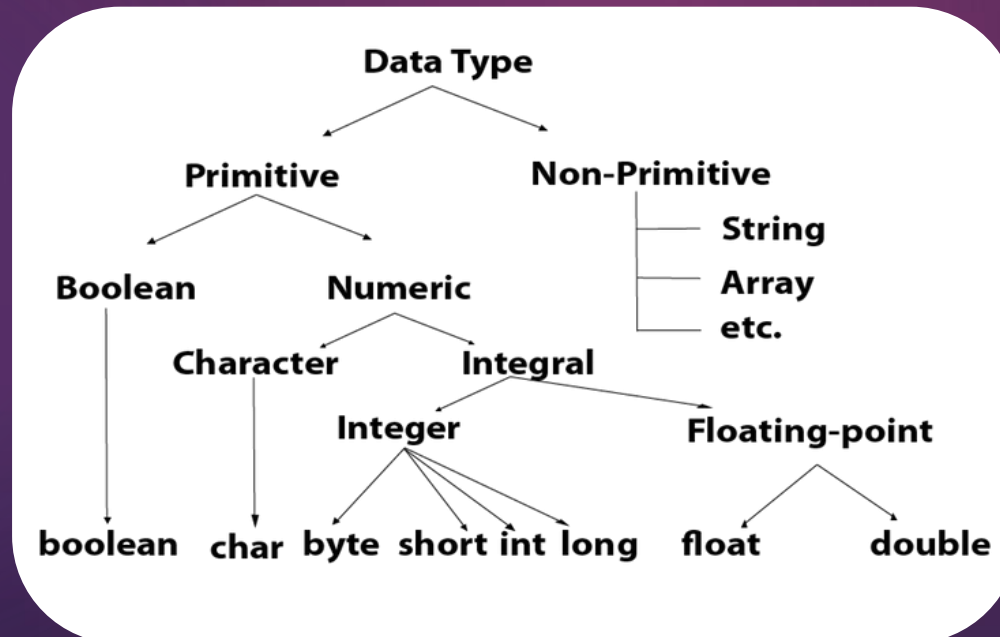
34] explain the data type available in java [8 marks]

[2018,2015,2014]

Data Types in Java

Data types specify the different sizes and values that can be stored in the variable. There are two types of data types in Java :

- 1.Primitive data types**: The primitive data types include Boolean, char, byte, short, int, long, float and double.
- 2.Non-primitive data types**: The non-primitive data types include Classes, Interfaces, and Arrays.



Boolean Data Type

The Boolean data type is used to store only two possible values: true and false. This data type is used for simple flags that track true/false conditions.

Byte Data Type

The byte data type is an example of primitive data type. It is an 8-bit signed two's complement integer. Its value-range lies between -128 to 127 (inclusive). Its minimum value is -128 and maximum value is 127. Its default value is 0.

Short Data Type

The short data type is a 16-bit signed two's complement integer. Its value-range lies between -32,768 to 32,767 (inclusive). Its minimum value is -32,768 and maximum value is 32,767. Its default value is 0.

Int Data Type

The int data type is a 32-bit signed two's complement integer. Its value-range lies between -2,147,483,648 (-2^{31}) to 2,147,483,647 ($2^{31} - 1$) (inclusive). Its minimum value is -2,147,483,648 and maximum value is 2,147,483,647. Its default value is 0.

Long Data Type

The long data type is a 64-bit two's complement integer. Its value-range lies between -9,223,372,036,854,775,808(-2^{63}) to 9,223,372,036,854,775,807($2^{63} - 1$)(inclusive). Its minimum value is -9,223,372,036,854,775,808 and maximum value is 9,223,372,036,854,775,807. Its default value is 0. The long data type is used when you need a range of values more than those provided by int.

Float Data Type

The float data type is a single-precision 32-bit IEEE 754 floating point. Its value range is unlimited. It is recommended to use a float (instead of double) if you need to save memory in large arrays of floating point numbers. The float data type should never be used for precise values, such as currency. Its default value is 0.0F.

Double Data Type

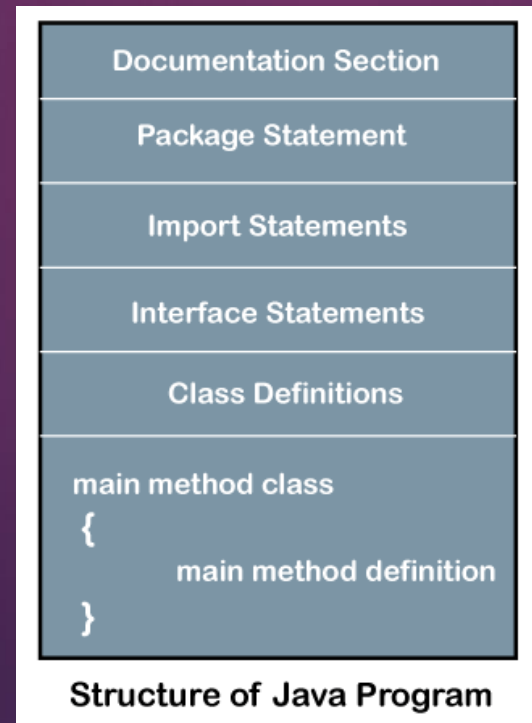
The double data type is a double-precision 64-bit IEEE 754 floating point. Its value range is unlimited. The double data type is generally used for decimal values just like float. The double data type also should never be used for precise values, such as currency. Its default value is 0.0d.

Char Data Type

The char data type is a single 16-bit Unicode character. Its value-range lies between '\u0000' (or 0) to '\uffff' (or 65,535 inclusive). The char data type is used to store characters.

Structure of Java Program

Java is an object-oriented programming, platform-independent, and **secure** programming language that makes it popular. Using the Java programming language, we can develop a wide variety of applications. So, before diving in depth, it is necessary to understand the basic structure of Java program in detail. In this section, we have discussed the basic **structure of a Java program**. At the end of this section, you will able to develop the Hello world Java program, easily.



- Documentation Section
- Package Declaration
- Import Statements
- Interface Section
- Class Definition
- Class Variables and Variables
- Main Method Class
- Methods and Behaviors

Documentation Section

- The documentation section is an important section but optional for a Java program.
- It includes basic information about a Java program.
- The information includes the author's name, date of creation, version, program name, company name, and description of the program.
- It improves the readability of the program.
- Whatever we write in the documentation section, the Java compiler ignores the statements during the execution of the program.
- To write the statements in the documentation section, we use comments.
- The comments may be single-line, multi-line, and documentation comments.

Package Declaration

- The package declaration is optional.
- It is placed just after the documentation section.
- In this section, we declare the **package name** in which the class is placed.
- We use the keyword package to declare the package name

Import Statements

- The package contains the many predefined classes and interfaces.
- If we want to use any class of a particular package, we need to import that class.
- The import statement represents the class stored in the other package.
- We use the import keyword to import the class.

Interface Section

- It is an optional section.
- We can create an **interface** in this section if required.
- We use the interface keyword to create an interface.

Class Definition

- In this section, we define the class.
- It is vital part of a Java program.
- Without the class, we cannot create any Java program.
- A Java program may contain more than one class definition.
- We use the class keyword to define the class.
- The class is a blueprint of a Java program.
- It contains information about user-defined methods, variables, and constants.
- Every Java program has at least one class that contains the main() method.

Class Variables and Constants

- In this section, we define variables and constants that are to be used later in the program.
- In a Java program, the variables and constants are defined just after the class definition.
- The variables and constants store values of the parameters.
- It is used during the execution of the program.

Main Method Class

- In this section, we define the main() method.
- It is essential for all Java programs. Because the execution of all Java programs starts from the main() method.

Bca expert

36] write a note on command line arguments [4 marks]

[2018, 2017,2016, 2011]

Java Command Line Arguments

- The java command-line argument is an argument i.e. passed at the time of running the java program.
- The arguments passed from the console can be received in the java program and it can be used as an input.
- So, it provides a convenient way to check the behavior of the program for the different values. You can pass **N** (1,2,3 and so on) numbers of arguments from the command prompt.

Simple example of command-line argument in java

In this example, we are receiving only one argument and printing it. To run this java program, you must pass at least one argument from the command prompt.

```
class Command Line Example
{
    public static void main(String args[])
    {
        System.out.println("Your first argument is: "+args[0]);
    }
}
```

37] list and explain java API packages [8 marks]

[2018,2015,2011]

- Package java. Applet
Provides the classes necessary to create an applet and the classes an applet uses to communicate with its applet context.
- Package java.awt
Contains all of the classes for creating user interfaces and for painting graphics and images.
- Package java.awt. Color
Provides classes for color spaces.
- Package java.awt.datatransfer
Provides interfaces and classes for transferring data between and within applications.
- Package java.awt.event
Provides interfaces and classes for dealing with different types of events fired by AWT components.
- Package java.awt.font
Provides classes and interface relating to fonts.
- Package java.awt.geom
Provides the Java 2D classes for defining and performing operations on objects related to two-dimensional geometry.

38] explain input output stream classes [8 marks]

[2018,2015]

Java I/O

- Java I/O (Input and Output) is used to process the input and produce the output .
- Java uses the concept of a stream to make I/O operation fast.
- The java.io package contains all the classes required for input and output operations.
- We can perform file handling in Java by Java I/O A

Stream

- A stream is a sequence of data. In Java, a stream is composed of bytes. It's called a stream because it is like a stream of water that continues to flow.

Output Stream vs Input Stream

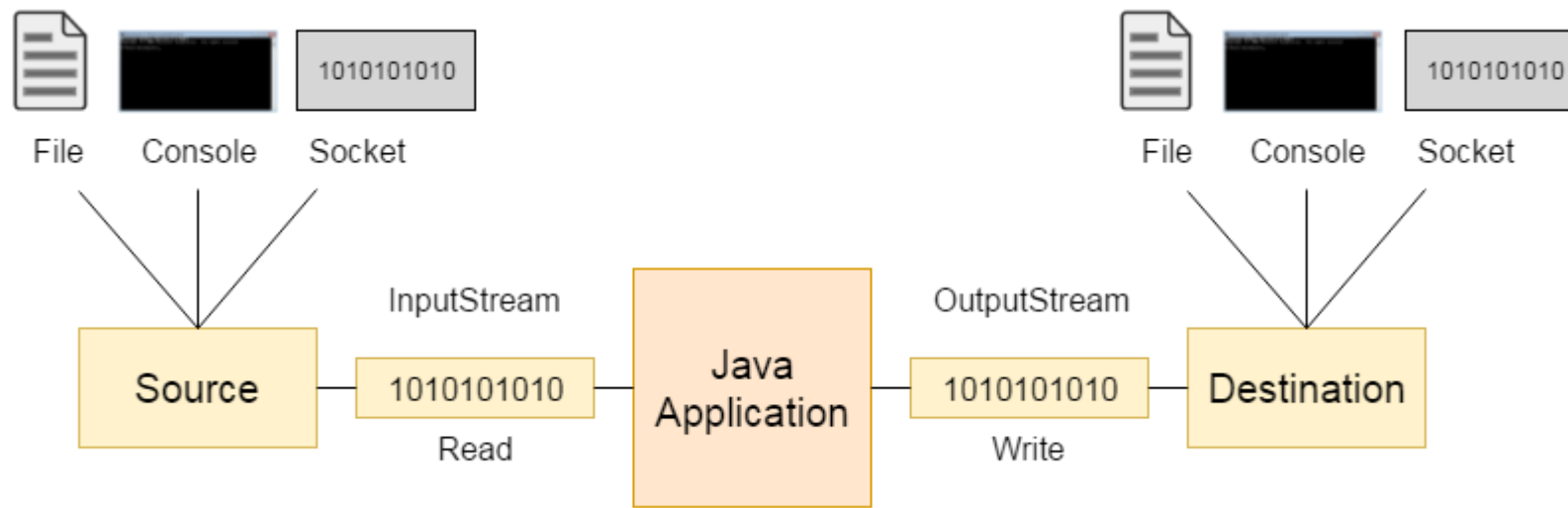
Output Stream

Java application uses an output stream to write data to a destination; it may be a file, an array, peripheral device or socket.

Input Stream

Java application uses an input stream to read data from a source; it may be a file, an array, peripheral device or socket.

Output Stream and Input Stream



39] write a java program to demonstrate string method [8 marks]

[2021,2017,2015,2016,2014]

Create a String in Java

```
class Main {  
    public static void main(String[] args) {  
  
        // create strings  
        String first = "Java";  
        String second = "Python";  
        String third = "JavaScript";  
  
        // print strings  
        System.out.println(first); // print Java  
        System.out.println(second); // print Python  
        System.out.println(third); // print JavaScript  
    }  
}
```

Output

```
String: Hello! World  
Length: 12
```

- In the above example, we have created three strings named first, second, and third. Here, we are directly creating strings like primitive types.

Bca expert

39] write short notes [4marks]

[2021,2019,2018, 2017,2016,2015,2014,2011]

1] Exception handling	[2011, 2016]		
2] awt package	[2011]		
3] wrapper classes	[2011,2014, 2016, 2019, 2021]		
4] labelled loop	[2011]		
5] bitwise operators	[2011]	21] java operators	[2017]
6] string buffer class	[2011]	22] constructors	[2017,2021]
7] vector class	[2014,2015, 2018]	23] I-O packages	[2017]
8] JVM	[2014, 2019]	24] java history	[2017,2018,2021]
9] looping statements	[2014]	25] difference between c and java	[2018]
10] final method	[2014]	26] finally	[2018]
11] java tokens	[2014, 2015, 2017]	27] HTML tag	[2019]
12] thread priority	[2015]		
13] final keyword	[2015]		
14] super	[2015]		
15] arrays	[2015, 2019]		
16] visibility control	[2016, 2018]		
17] graphical programming	[2016]		
18] string method	[2016]		
19] interface	[2016]		
20] application of oops	[2017]		

Wrapper classes in Java

The wrapper class in Java provides the mechanism to convert primitive into object and object into primitive.

Use of Wrapper classes in Java

Java is an object-oriented programming language, so we need to deal with objects many times like in Collections, Serialization, Synchronization, etc. Let us see the different scenarios, where we need to use the wrapper classes.

- Change the value in Method : Java supports only call by value. So, if we pass a primitive value, it will not change the original value. But, if we convert the primitive value in an object, it will change the original value.
- Serialization : We need to convert the objects into streams to perform the serialization. If we have a primitive value, we can convert it in objects through the wrapper classes.
- Synchronization : Java synchronization works with objects in Multithreading.
- java.util package : The java.util package provides the utility classes to deal with objects.
- Collection Framework : Java collection framework works with objects only. All classes of the collection framework (ArrayList, LinkedList, Vector, HashSet, LinkedHashSet, TreeSet, PriorityQueue, ArrayDeque, etc.) deal with objects only.

Java Vector

- Vector is like the dynamic array which can grow or shrink its size.
- Unlike array, we can store n-number of elements in it as there is no size limit.
- It is a part of Java Collection framework since Java 1.2. It is found in the java.util package and implements the List interface, so we can use all the methods of List interface here.
- It is recommended to use the Vector class in the thread-safe implementation only.
- If you don't need to use the thread-safe implementation, you should use the ArrayList, the ArrayList will perform better in such case.
- The Iterators returned by the Vector class are fail-fast.
- In case of concurrent modification, it fails and throws the Concurrent Modification Exception.
- It is similar to the ArrayList, but with two differences-
 - Vector is synchronized.
 - Java Vector contains many legacy methods that are not the part of a collections framework.

Java Vector class Declaration

```
public class Vector<E>  
extends Object<E>  
implements List<E>, Cloneable, Serializable
```

Java Tokens

- In Java, the program contains classes and methods.
- Further, the methods contain the expressions and statements required to perform a specific operation. These statements and expressions are made up of **tokens**.
- In other words, we can say that the expression and statement is a set of **tokens**.
- The tokens are the small building blocks of a Java program that are meaningful to the Java compiler. Further, these two components contain variables, constants, and operators.
- In this section, we will discuss **what is tokens in Java**.

What is token in Java?

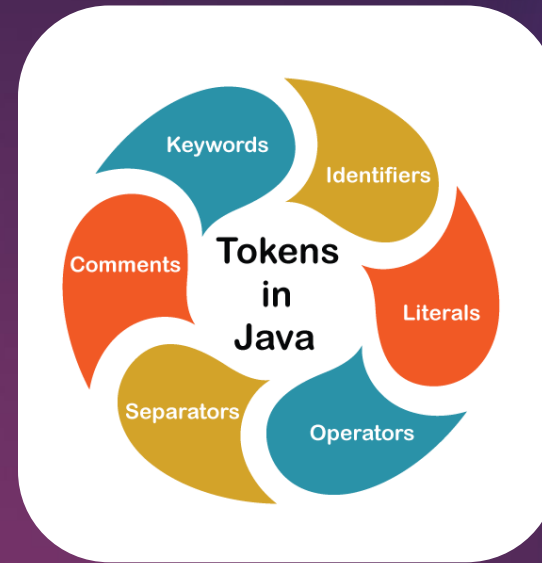
- The Java compiler breaks the line of code into text (words) is called **Java tokens**.
- These are the smallest element of the Java program.
- The Java compiler identified these words as tokens. These tokens are separated by the delimiters.
- It is useful for compilers to detect errors.
- Remember that the delimiters are not part of the Java tokens.

➤ token <= identifier | keyword | separator | operator | literal | comment

Types of Tokens

Java token includes the following :

1. Keywords
2. Identifiers
3. Literals
4. Operators
5. Separators
6. Comments



- **Keywords** : These are the **pre-defined** reserved words of any programming language. Each keyword has a special meaning. It is always written in lower case.
- **Identifier** : Identifiers are used to name a variable, constant, function, class, and array. It usually defined by the user.
- **Literals** : In programming literal is a notation that represents a fixed value (constant) in the source code. It can be categorized as an integer literal, string literal, Boolean literal, etc. It is defined by the programmer
- **Operators** : In programming, operators are the special symbol that tells the compiler to perform a special operation. Java provides different types of operators that can be classified according to the functionality they provide.
- **Separators** : The separators in Java is also known as **punctuators**.
- **Comments** : Comments allow us to specify information about the program inside our Java code. Java compiler recognizes these comments as tokens but excludes it from further processing

History of Java

- The history of Java is very interesting.
- Java was originally designed for interactive television, but it was too advanced technology for the digital cable television industry at the time.
- The history of Java starts with the Green Team.
- Java team members (also known as Green Team), initiated this project to develop a language for digital devices such as set-top boxes, televisions, etc.
- Currently, Java is used in internet programming, mobile devices, games, e-business solutions, etc. Following are given significant points that describe the history of Java.
- James Gosling
 - Mike Sheridan, and Patrick Naughton initiated the Java language project in June 1991.
 - The small team of sun engineers called Green Team.
- Initially it was designed for small, Embedded system
 - in electronic appliances like set-top boxes.
- Firstly, it was called "Green talk" by James Gosling, and the file extension was .gt.
- After that, it was called **Oak** and was developed as a part of the Green project.
- In 1995, Oak was renamed as "Java" because it was already a trademark by Oak Technologies.
- In 1995, Time magazine called Java one of the Ten Best Products of 1995.

Exception Handling in Java

- The Exception Handling in Java is one of the powerful mechanism to handle the runtime errors so that the normal flow of the application can be maintained.

What is Exception in Java?

In Java, an exception is an event that disrupts the normal flow of the program. It is an object which is thrown at runtime.

What is Exception Handling?

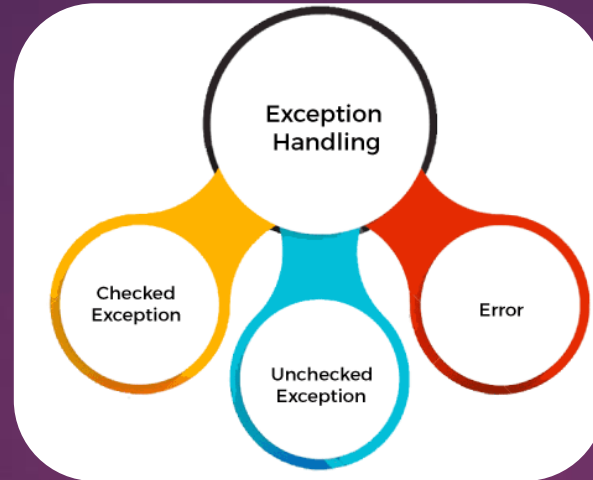
Exception Handling is a mechanism to handle runtime errors such as Class Not Found Exception, IO Exception, SQL Exception, Remote Exception, etc.

Advantage of Exception Handling

- The core advantage of exception handling is to maintain the normal flow of the application. An exception normally disrupts the normal flow of the application; that is why we need to handle exceptions

Types of Java Exceptions

There are mainly two types of exceptions: checked and unchecked. An error is considered as the unchecked exception. However, according to Oracle, there are three types of exceptions namely:



1. Checked Exception
2. Unchecked Exception
3. Error

1) Checked Exception

The classes that directly inherit the Throwable class except RuntimeException and Error are known as checked exceptions. For example, IOException, SQLException, etc. Checked exceptions are checked at compile-time.

2) Unchecked Exception

The classes that inherit the RuntimeException are known as unchecked exceptions. For example, ArithmeticException, NullPointerException, ArrayIndexOutOfBoundsException, etc. Unchecked exceptions are not checked at compile-time, but they are checked at runtime.

3) Error

Error is irrecoverable. Some example of errors are OutOfMemoryError, VirtualMachineError, AssertionError etc.

JVM (Java Virtual Machine) Architecture

- JVM (Java Virtual Machine) is an abstract machine.
- It is a specification that provides runtime environment in which java bytecode can be executed.
- JVMs are available for many hardware and software platforms (i.e. JVM is platform dependent).

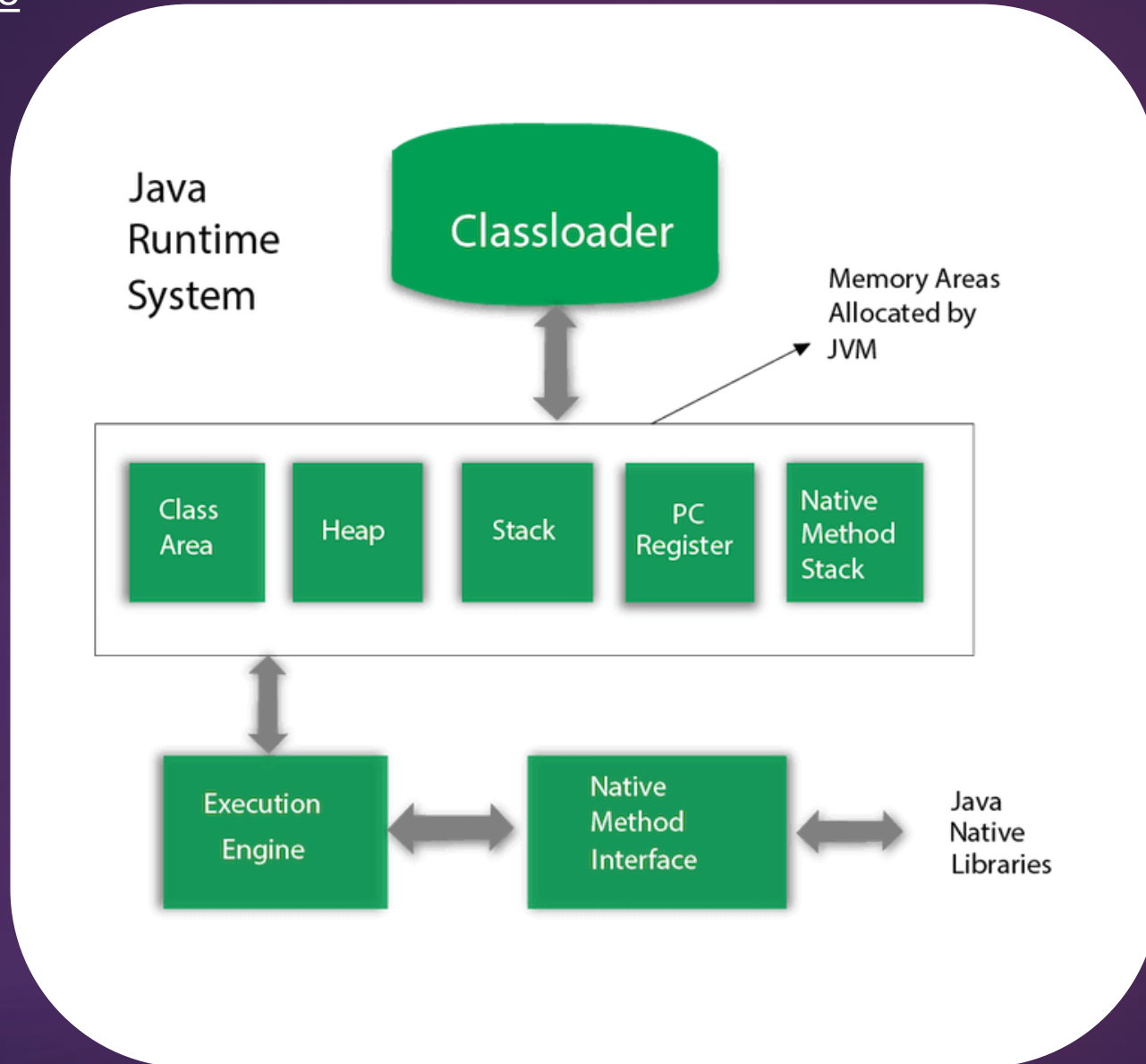
What is JVM

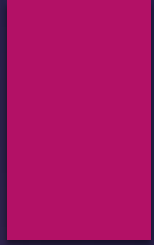
1] A specification where working of Java Virtual Machine is specified. But implementation provider is independent to choose the algorithm. Its implementation has been provided by Oracle and other companies.

2] An implementation Its implementation is known as JRE (Java Runtime Environment).

3] Runtime Instance Whenever you write java command on the command prompt to run the java class, an instance of JVM is created.

JVM Architecture





1) Class loader

- Class loader is a subsystem of JVM which is used to load class files.
- Whenever we run the java program, it is loaded first by the class loader.
- There are three built-in class loaders in Java.
- Bootstrap Class Loader : This is the first class loader which is the super class of Extension class loader.
- Extension Class Loader : This is the child class loader of Bootstrap and parent class loader of System class loader.
- System/Application Class Loader : This is the child class loader of Extension class loader.
- It loads the class files from class path.
- By default, class path is set to current directory.
- You can change the class path using "-cp" or "-class path" switch.
- It is also known as Application class loader.

2) Class(Method) Area

- Class(Method) Area stores per-class structures such as the runtime constant pool, field and method data, the code for methods.

3) Heap

- It is the runtime data area in which objects are allocated.

4) Stack

- Java Stack stores frames. It holds local variables and partial results, and plays a part in method invocation and return.
- Each thread has a private JVM stack, created at the same time as thread.

5) Program Counter Register

- PC (program counter) register contains the address of the Java virtual machine instruction currently being executed.

6) Native Method Stack

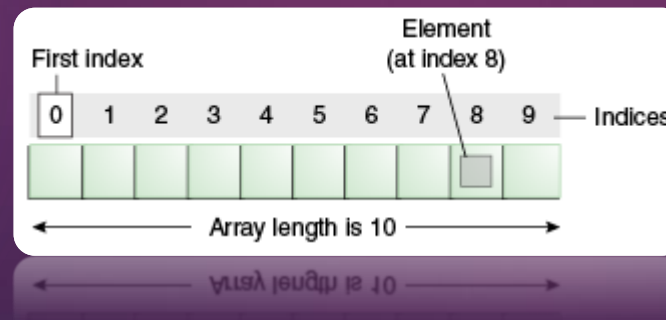
- It contains all the native methods used in the application.

7) Java Native Interface

- Java Native Interface (JNI) is a framework which provides an interface to communicate with another application written in another language like C, C++, Assembly etc. Java uses JNI framework to send output to the Console or interact with OS libraries.

Java Arrays

- Normally, an array is a collection of similar type of elements which has contiguous memory location.
- Java array is an object which contains elements of a similar data type. Additionally, The elements of an array are stored in a contiguous memory location.
- It is a data structure where we store similar elements.
- We can store only a fixed set of elements in a Java array.
- Array in Java is index-based, the first element of the array is stored at the 0th index, 2nd element is stored on 1st index and so on.
- In Java, array is an object of a dynamically generated class.
- Java array inherits the Object class, and implements the Serializable as well as Cloneable interfaces.
- We can store primitive values or objects in an array in Java.



Advantages

- Code Optimization : It makes the code optimized, we can retrieve or sort the data efficiently.
- Random access: We can get any data located at an index position.

Disadvantages

- Size Limit : We can store only the fixed size of elements in the array.
- It doesn't grow its size at runtime.
- To solve this problem, collection framework is used in Java which grows automatically.

Types of Array in java

There are two types of array.

- Single Dimensional Array
- Multidimensional Array

visibility modes

The access modifiers in Java specify the accessibility or scope of a field, method, constructor, or class. We can change the access level of fields, constructors, methods, and class by applying the access modifier on it.

1.Private : The access level of a private modifier is only within the class. It cannot be accessed from outside the class.

2.Default : The access level of a default modifier is only within the package. It cannot be accessed from outside the package. If you do not specify any access level, it will be the default.

3.Protected : The access level of a protected modifier is within the package and outside the package through child class. If you do not make the child class, it cannot be accessed from outside the package.

4.Public : The access level of a public modifier is everywhere. It can be accessed from within the class, outside the class, within the package and outside the package.

Access Modifier	within class	within package	outside package by subclass only	outside package
Private	Y	N	N	N
Default	Y	Y	N	N
Protected	Y	Y	Y	N
Public	Y	Y	Y	Y

Constructors in Java

- In java a constructor is a block of codes similar to the method.
- It is called when an instance of the class is created.
- At the time of calling constructor, memory for the object is allocated in the memory.
- It is a special type of method which is used to initialize the object.
- Every time an object is created using the new() keyword, at least one constructor is called.
- It calls a default constructor if there is no constructor available in the class.
- In such case, Java compiler provides a default constructor by default.

Rules for creating Java constructor

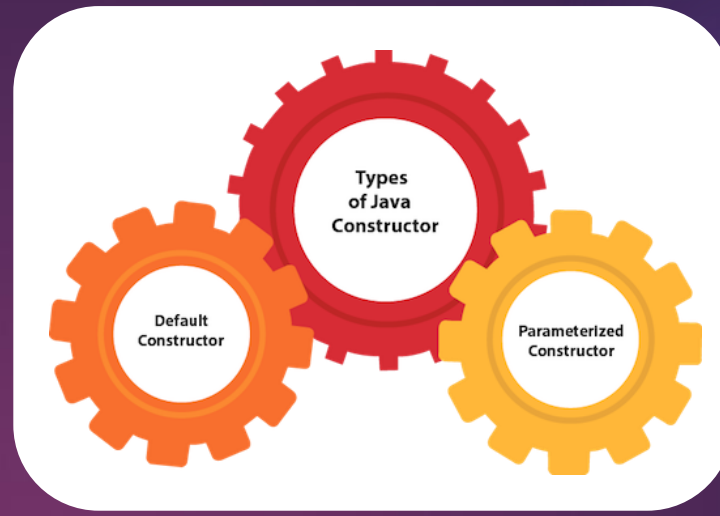
There are two rules defined for the constructor.

1. Constructor name must be the same as its class name
2. A Constructor must have no explicit return type
3. A Java constructor cannot be abstract, static, final, and synchronized

Types of Java constructors

There are two types of constructors in Java:

1. Default constructor (no-arg constructor)
2. Parameterized constructor



1] Java Default Constructor

A constructor is called "Default Constructor" when it doesn't have any parameter.

The default constructor is used to provide the default values to the object like 0, null, etc., depending on the type.

Syntax of default constructor:

1.<class_name>(){}

2] Java Parameterized Constructor

A constructor which has a specific number of parameters is called a parameterized constructor.

The parameterized constructor is used to provide different values to distinct objects. However, you can provide the same values also.

